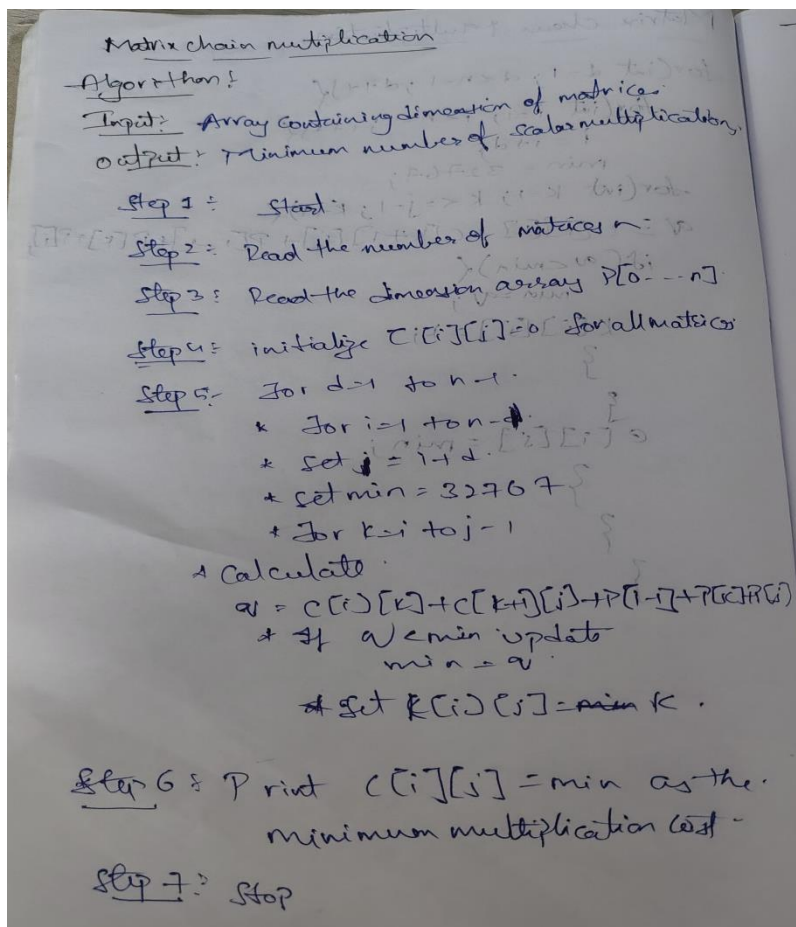


PRACTICAL-6

AIM: Implementation of chain matrix multiplication using dynamic programming.

ALGORITHM:



CODE:

```
#include <iostream>
#include <climits>
using namespace std;
```

```
int main()
{
```

```
int n;

cout << "Enter number of matrices: ";
cin >> n;

int p[n + 1];

cout << "Enter dimensions of matrices: ";
for (int i = 0; i <= n; i++)
{
    cin >> p[i];
}

int m[n + 1][n + 1];

for (int i = 1; i <= n; i++)
{
    m[i][i] = 0;
}

for (int L = 2; L <= n; L++)
{
    for (int i = 1; i <= n - L + 1; i++)
    {
        int j = i + L - 1;

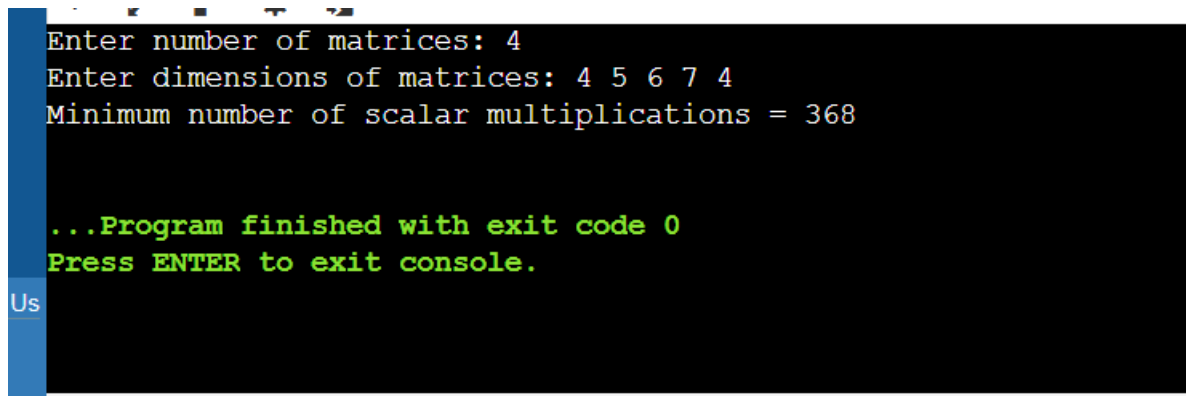
        m[i][j] = INT_MAX;

        for (int k = i; k < j; k++)
        {
            int cost = m[i][k] +
                        m[k + 1][j] +
                        p[i - 1] * p[k] * p[j];

            if (cost < m[i][j])
            {
                m[i][j] = cost;
            }
        }
    }
}
```

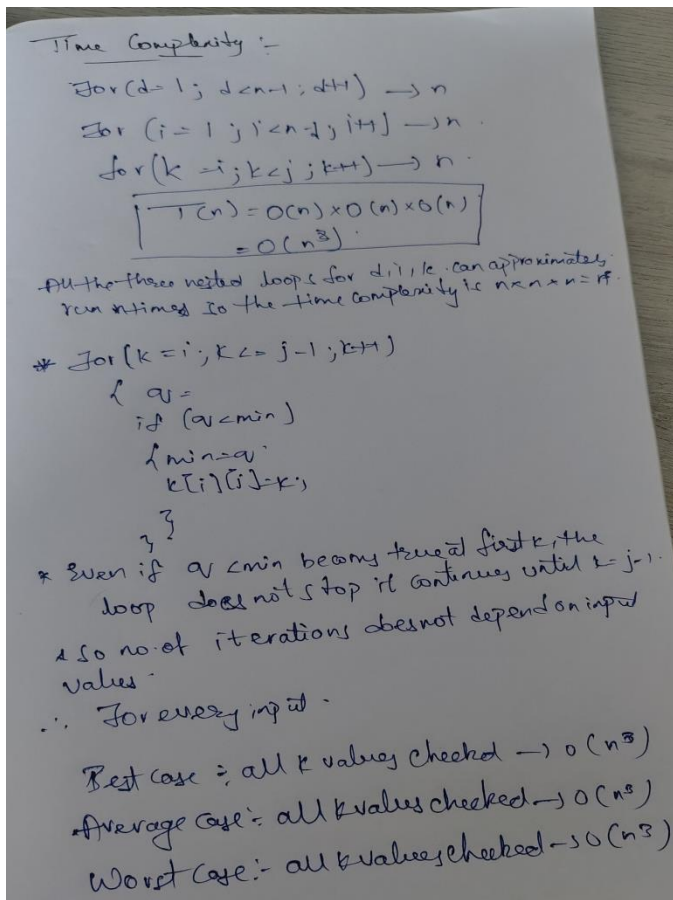
```
cout << "Minimum number of scalar multiplications = "  
    << m[1][n] << endl;  
  
return 0;  
}
```

OUTPUT:



```
Enter number of matrices: 4  
Enter dimensions of matrices: 4 5 6 7 4  
Minimum number of scalar multiplications = 368  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

TIME COMPLEXITY:



CONCLUSION:

The Matrix Chain Multiplication problem was successfully implemented using Dynamic Programming. The algorithm finds the optimal order of matrix multiplication with $O(n^3)$ time complexity and $O(n^2)$ space complexity, reducing unnecessary repeated calculations.